# System Verilog Assignment

Name : Kumar Dipti Manna
Roll : CrS2509

## Assignment 1: Full Adder Design

For the full adder I used assign statements. The sum is just a XOR b XOR c, and carry is high when at least two of the inputs are 1, so carry = ab + bc + ac. In the testbench I gave all 8 combinations of a, b and c, with 5 time units between each. One thing I noticed is that the given code didn't connect c when creating the `fulladder1` instance, so I added it.

### Code

```
module fulladder1(a, b, c, sum, carry);
    input a, b, c;
    output sum, carry;
    wire sum, carry;
    assign sum   = a ^ b ^ c;
    assign carry = (a & b) | (b & c) | (a & c);
endmodule

module main();
    reg  a, b, c;
    wire sum1, carry1;

    fulladder1 add1(a, b, c, sum1, carry1);

    initial
    begin
        $monitor("%b + %b + %b = %b, carry = %b", a, b, c, sum1, carry1);
                a = 0; b = 0; c = 0;
        #5    a = 0; b = 0; c = 1;
        #5    a = 0; b = 1; c = 0;
        #5    a = 0; b = 1; c = 1;
        #5    a = 1; b = 0; c = 0;
        #5    a = 1; b = 0; c = 1;
        #5    a = 1; b = 1; c = 0;
        #5    a = 1; b = 1; c = 1;
    end
endmodule
```

### Simulation Output

```
0 + 0 + 0 = 0, carry = 0
0 + 0 + 1 = 1, carry = 0
0 + 1 + 0 = 1, carry = 0
0 + 1 + 1 = 0, carry = 1
1 + 0 + 0 = 1, carry = 0
1 + 0 + 1 = 0, carry = 1
1 + 1 + 0 = 0, carry = 1
1 + 1 + 1 = 1, carry = 1
```

## Assignment 2: Multiplexer Design (2:1)

This is a 2:1 mux. If `sel` is 0 the output is `datain_0`, and if `sel` is 1 the output is `datain_1`. I wrote it with a single assign statement using the conditional operator. In the testbench I went through all 8 combinations of `d0`, `d1` and `sel`, and the output matched what I expected each time.

## Code

```
module mux1(datain_0, datain_1, sel, mux_out);
    input datain_0, datain_1, sel;
    output mux_out;
    wire mux_out;
    assign mux_out = sel ? datain_1 : datain_0;
endmodule

module main();
    reg  d0, d1, sel;
    wire out1;

    mux1 m1(d0, d1, sel, out1);

    initial
    begin
        $monitor("d0=%b d1=%b sel=%b -> mux_out = %b", d0, d1, sel, out1);
                d0 = 0; d1 = 0; sel = 0;
        #5    d0 = 0; d1 = 0; sel = 1;
        #5    d0 = 0; d1 = 1; sel = 0;
        #5    d0 = 0; d1 = 1; sel = 1;
        #5    d0 = 1; d1 = 0; sel = 0;
        #5    d0 = 1; d1 = 0; sel = 1;
        #5    d0 = 1; d1 = 1; sel = 0;
        #5    d0 = 1; d1 = 1; sel = 1;
    end
endmodule
```

## Simulation Output

```
d0=0 d1=0 sel=0 -> mux_out = 0
d0=0 d1=0 sel=1 -> mux_out = 0
d0=0 d1=1 sel=0 -> mux_out = 0
d0=0 d1=1 sel=1 -> mux_out = 1
d0=1 d1=0 sel=0 -> mux_out = 1
d0=1 d1=0 sel=1 -> mux_out = 0
d0=1 d1=1 sel=0 -> mux_out = 1
d0=1 d1=1 sel=1 -> mux_out = 1
```

## Assignment 3: Decoder Using Gate Primitives

2-to-4 decoder with enable built only from `not` and `and` primitives.

### Code

```
// 2-to-4 decoder with enable, gate primitives only
module decoder_gate(en, a, b, y);
    input en, a, b;
    output [3:0] y;
    wire na, nb;
    not (na, a);
    not (nb, b);
    and (y[0], en, na, nb);
    and (y[1], en, na, b );
    and (y[2], en, a,  nb);
    and (y[3], en, a,  b );
endmodule

module main();
    reg en, a, b;
    wire [3:0] y;
    integer i;

    decoder_gate d1(en, a, b, y);

    initial
    begin
        $monitor("en=%b a=%b b=%b -> y=%b", en, a, b, y);
        for (i = 0; i < 8; i = i + 1)
        begin
            {en, a, b} = i;
            #5;
        end
    end
endmodule
```

### Simulation Output

```
en=0 a=0 b=0 -> y=0000
en=0 a=0 b=1 -> y=0000
en=0 a=1 b=0 -> y=0000
en=0 a=1 b=1 -> y=0000
en=1 a=0 b=0 -> y=0001
en=1 a=0 b=1 -> y=0010
en=1 a=1 b=0 -> y=0100
en=1 a=1 b=1 -> y=1000
```

## Assignment 4: Decoder Using Assign

Same 2-to-4 decoder with enable, written with continuous `assign` statements.

### Code

```
// 2-to-4 decoder with enable, continuous assignments
module decoder_assign(en, a, b, y);
    input en, a, b;
    output [3:0] y;
    assign y[0] = en & ~a & ~b;
    assign y[1] = en & ~a &  b;
    assign y[2] = en &  a & ~b;
```

```
    assign y[3] = en &  a &  b;
endmodule

module main();
    reg en, a, b;
    wire [3:0] y;
    integer i;

    decoder_assign d1(en, a, b, y);

    initial
    begin
        $monitor("en=%b a=%b b=%b -> y=%b", en, a, b, y);
        for (i = 0; i < 8; i = i + 1)
        begin
            {en, a, b} = i;
            #5;
        end
    end
endmodule
```

## Simulation Output

```
en=0 a=0 b=0 -> y=0000
en=0 a=0 b=1 -> y=0000
en=0 a=1 b=0 -> y=0000
en=0 a=1 b=1 -> y=0000
en=1 a=0 b=0 -> y=0001
en=1 a=0 b=1 -> y=0010
en=1 a=1 b=0 -> y=0100
en=1 a=1 b=1 -> y=1000
```

## Assignment 5: Complex Multiplexer (4:1, Array)

4-bit data, 2-bit select, 1-bit output. The four data bits are stored in a `reg` array (`arr[0:3]`) and the output is `arr[sel]`. The testbench loops over all $16 \times 4 = 64$ combinations.

### Code

```
module mux4(data, sel, mux_out);
    input  [3:0] data;
    input  [1:0] sel;
    output       mux_out;
    reg arr [0:3];              // array of 4 elements, 1 bit each
    always @(data) begin
        arr[0] = data[0];
        arr[1] = data[1];
        arr[2] = data[2];
        arr[3] = data[3];
    end
    assign mux_out = arr[sel]; // pick element using sel
endmodule

module main();
    reg  [3:0] data;
    reg  [1:0] sel;
    wire       out1;
    integer i;

    mux4 m1(data, sel, out1);

    initial
    begin
        $monitor("data=%b sel=%b -> mux_out=%b", data, sel, out1);
        for (i = 0; i < 64; i = i + 1)   // all 16 data x 4 sel combinations
        begin
            {data, sel} = i;
            #5;
        end
    end
endmodule
```

### Simulation Output

```
data=0000 sel=00 -> mux_out=0
data=0000 sel=01 -> mux_out=0
data=0000 sel=10 -> mux_out=0
data=0000 sel=11 -> mux_out=0
data=0001 sel=00 -> mux_out=1
data=0001 sel=01 -> mux_out=0
data=0001 sel=10 -> mux_out=0
data=0001 sel=11 -> mux_out=0
data=0010 sel=00 -> mux_out=0
data=0010 sel=01 -> mux_out=1
data=0010 sel=10 -> mux_out=0
data=0010 sel=11 -> mux_out=0
data=0011 sel=00 -> mux_out=1
data=0011 sel=01 -> mux_out=1
data=0011 sel=10 -> mux_out=0
data=0011 sel=11 -> mux_out=0
data=0100 sel=00 -> mux_out=0
data=0100 sel=01 -> mux_out=0
data=0100 sel=10 -> mux_out=1
data=0100 sel=11 -> mux_out=0
data=0101 sel=00 -> mux_out=1
data=0101 sel=01 -> mux_out=0
data=0101 sel=10 -> mux_out=1
data=0101 sel=11 -> mux_out=0
data=0110 sel=00 -> mux_out=0
data=0110 sel=01 -> mux_out=1
data=0110 sel=10 -> mux_out=1
data=0110 sel=11 -> mux_out=0
data=0111 sel=00 -> mux_out=1
data=0111 sel=01 -> mux_out=1
data=0111 sel=10 -> mux_out=1
data=0111 sel=11 -> mux_out=0
data=1000 sel=00 -> mux_out=0
data=1000 sel=01 -> mux_out=0
data=1000 sel=10 -> mux_out=0
data=1000 sel=11 -> mux_out=1
data=1001 sel=00 -> mux_out=1
data=1001 sel=01 -> mux_out=0
data=1001 sel=10 -> mux_out=0
data=1001 sel=11 -> mux_out=1
data=1010 sel=00 -> mux_out=0
data=1010 sel=01 -> mux_out=1
data=1010 sel=10 -> mux_out=0
data=1010 sel=11 -> mux_out=1
data=1011 sel=00 -> mux_out=1
data=1011 sel=01 -> mux_out=1
data=1011 sel=10 -> mux_out=0
```

```
data=1011 sel=11 -> mux_out=1
data=1100 sel=00 -> mux_out=0
data=1100 sel=01 -> mux_out=0
data=1100 sel=10 -> mux_out=1
data=1100 sel=11 -> mux_out=1
data=1101 sel=00 -> mux_out=1
data=1101 sel=01 -> mux_out=0
data=1101 sel=10 -> mux_out=1
data=1101 sel=11 -> mux_out=1
```

```
data=1110 sel=00 -> mux_out=0
data=1110 sel=01 -> mux_out=1
data=1110 sel=10 -> mux_out=1
data=1110 sel=11 -> mux_out=1
data=1111 sel=00 -> mux_out=1
data=1111 sel=01 -> mux_out=1
data=1111 sel=10 -> mux_out=1
data=1111 sel=11 -> mux_out=1
```

## Assignment 6: 4-bit Gray to Binary

$b_3 = g_3,\ b_2 = b_3 \oplus g_2,\ b_1 = b_2 \oplus g_1,\ b_0 = b_1 \oplus g_0$, implemented with `xor` gates. Output matches the given truth table for all 16 inputs.

### Code

```
// 4-bit Gray to Binary converter using XOR gates
// b3 = g3 ; b2 = b3^g2 ; b1 = b2^g1 ; b0 = b1^g0
module GraytoBin1(g, b);
    input  [3:0] g;   // g[3] = MSB ... g[0] = LSB
    output [3:0] b;
    assign b[3] = g[3];
    xor (b[2], b[3], g[2]);
    xor (b[1], b[2], g[1]);
    xor (b[0], b[1], g[0]);
endmodule

module main();
    reg  [3:0] g;
    wire [3:0] b;
    integer i;

    GraytoBin1 u1(g, b);

    initial
    begin
        $monitor("Gray = %b  ->  Binary = %b", g, b);
        for (i = 0; i < 16; i = i + 1)
        begin
            g = i;
            #5;
        end
    end
endmodule
```

### Simulation Output

```
Gray = 0000  ->  Binary = 0000
Gray = 0001  ->  Binary = 0001
Gray = 0010  ->  Binary = 0011
Gray = 0011  ->  Binary = 0010
Gray = 0100  ->  Binary = 0111
Gray = 0101  ->  Binary = 0110
Gray = 0110  ->  Binary = 0100
Gray = 0111  ->  Binary = 0101
Gray = 1000  ->  Binary = 1111
Gray = 1001  ->  Binary = 1110
Gray = 1010  ->  Binary = 1100
Gray = 1011  ->  Binary = 1101
Gray = 1100  ->  Binary = 1000
Gray = 1101  ->  Binary = 1001
Gray = 1110  ->  Binary = 1011
Gray = 1111  ->  Binary = 1010
```

# Assignment 7: D Flip-Flops and Latch

Eight D flip-flop variants (rising/falling edge × simple, synchronous reset, asynchronous reset active-high, asynchronous reset active-low) and a level-sensitive latch, all driven from one testbench. Reset is applied at $t = 117$: the asynchronous versions clear immediately, the synchronous ones at the next active clock edge. The latch follows `d` while `en`=1 and holds when `en`=0.

## Code

```
`define DELAY #2 //Flip-flop delay

// 1. Rising edge, simple
module dff_pos (d, clk, q);
    input d, clk; output q; reg q;
    always @ (posedge clk) q <= `DELAY d;
endmodule

// 2. Rising edge, synchronous reset (active high)
module dff_pos_sync (d, clk, reset, q);
    input d, clk, reset; output q; reg q;
    always @ (posedge clk)
        if (reset) q <= 0; else q <= `DELAY d;
endmodule

// 3. Rising edge, asynchronous reset, active high
module dff_pos_async_hi (d, clk, reset, q);
    input d, clk, reset; output q; reg q;
    always @ (posedge clk or posedge reset)
        if (reset) q <= 0; else q <= `DELAY d;
endmodule

// 4. Rising edge, asynchronous reset, active low
module dff_pos_async_lo (d, clk, reset_n, q);
    input d, clk, reset_n; output q; reg q;
    always @ (posedge clk or negedge reset_n)
        if (!reset_n) q <= 0; else q <= `DELAY d;
endmodule

// 5. Falling edge, simple
module dff_neg (d, clk, q);
    input d, clk; output q; reg q;
    always @ (negedge clk) q <= `DELAY d;
endmodule

// 6. Falling edge, synchronous reset (active high)
module dff_neg_sync (d, clk, reset, q);
    input d, clk, reset; output q; reg q;
    always @ (negedge clk)
        if (reset) q <= 0; else q <= `DELAY d;
endmodule

// 7. Falling edge, asynchronous reset, active high
module dff_neg_async_hi (d, clk, reset, q);
    input d, clk, reset; output q; reg q;
    always @ (negedge clk or posedge reset)
        if (reset) q <= 0; else q <= `DELAY d;
endmodule

// 8. Falling edge, asynchronous reset, active low
module dff_neg_async_lo (d, clk, reset_n, q);
    input d, clk, reset_n; output q; reg q;
    always @ (negedge clk or negedge reset_n)
        if (!reset_n) q <= 0; else q <= `DELAY d;
endmodule

// 9. Latch (transparent when en = 1)
module latch (d, en, q);
    input d, en; output q; reg q;
    always @ (d or en)
        if (en) q = d;
endmodule

module main();
```

```
    reg d, clk, reset, en;
    wire reset_n = ~reset;
    wire q1, q2, q3, q4, q5, q6, q7, q8, q9;

    dff_pos           u1(d, clk, q1);
    dff_pos_sync      u2(d, clk, reset, q2);
    dff_pos_async_hi  u3(d, clk, reset, q3);
    dff_pos_async_lo  u4(d, clk, reset_n, q4);
    dff_neg           u5(d, clk, q5);
    dff_neg_sync      u6(d, clk, reset, q6);
    dff_neg_async_hi  u7(d, clk, reset, q7);
    dff_neg_async_lo  u8(d, clk, reset_n, q8);
    latch             u9(d, en, q9);

    always #10 clk = ~clk;          // clock period = 20

    initial
    begin
        $display("time clk d rst en | pos pos_s pos_ah pos_al | neg neg_s neg_ah neg_al | latch");
        $monitor("%4t  %b  %b  %b   %b  |  %b    %b     %b      %b   |  %b    %b     %b      %b   |  %b",
                 $time, clk, d, reset, en, q1, q2, q3, q4, q5, q6, q7, q8, q9);
        clk = 0; d = 0; reset = 1; en = 0;
        #25 reset = 0;
        #10 d = 1; en = 1;        // d high, latch transparent
        #30 en = 0;
        #5  d = 0;                // latch holds, flops wait for edge
        #30 d = 1;
        #17 reset = 1;            // async resets act immediately, sync ones at next edge
        #25 reset = 0; en = 1;
        #20 d = 0;
        #40 $finish;
    end
endmodule
```

### Simulation Output

```
time clk d rst en | pos pos_s pos_ah pos_al | neg neg_s neg_ah neg_al | latch
   0  0  0  1   0  |  x    x     0      0    |  x    0     0      0    |  x
   2  0  0  1   0  |  x    x     0      0    |  0    0     0      0    |  x
  10  1  0  1   0  |  x    0     0      0    |  0    0     0      0    |  x
  12  1  0  1   0  |  0    0     0      0    |  0    0     0      0    |  x
  20  0  0  1   0  |  0    0     0      0    |  0    0     0      0    |  x
  25  0  0  0   0  |  0    0     0      0    |  0    0     0      0    |  x
  30  1  0  0   0  |  0    0     0      0    |  0    0     0      0    |  x
  35  1  1  0   1  |  0    0     0      0    |  0    0     0      0    |  1
  40  0  1  0   1  |  0    0     0      0    |  0    0     0      0    |  1
  42  0  1  0   1  |  0    0     0      0    |  1    1     1      1    |  1
  50  1  1  0   1  |  0    0     0      0    |  1    1     1      1    |  1
  52  1  1  0   1  |  1    1     1      1    |  1    1     1      1    |  1
  60  0  1  0   1  |  1    1     1      1    |  1    1     1      1    |  1
  65  0  1  0   0  |  1    1     1      1    |  1    1     1      1    |  1
  70  1  0  0   0  |  1    1     1      1    |  1    1     1      1    |  1
  72  1  0  0   0  |  0    0     0      0    |  1    1     1      1    |  1
  80  0  0  0   0  |  0    0     0      0    |  1    1     1      1    |  1
  82  0  0  0   0  |  0    0     0      0    |  0    0     0      0    |  1
  90  1  0  0   0  |  0    0     0      0    |  0    0     0      0    |  1
 100  0  1  0   0  |  0    0     0      0    |  0    0     0      0    |  1
 102  0  1  0   0  |  0    0     0      0    |  1    1     1      1    |  1
 110  1  1  0   0  |  0    0     0      0    |  1    1     1      1    |  1
 112  1  1  0   0  |  1    1     1      1    |  1    1     1      1    |  1
 117  1  1  1   0  |  1    1     0      0    |  1    1     0      0    |  1
 120  0  1  1   0  |  1    1     0      0    |  1    0     0      0    |  1
 130  1  1  1   0  |  1    0     0      0    |  1    0     0      0    |  1
 140  0  1  1   0  |  1    0     0      0    |  1    0     0      0    |  1
 142  0  1  0   1  |  1    0     0      0    |  1    0     0      0    |  1
 150  1  1  0   1  |  1    0     0      0    |  1    0     0      0    |  1
 152  1  1  0   1  |  1    1     1      1    |  1    0     0      0    |  1
 160  0  1  0   1  |  1    1     1      1    |  1    0     0      0    |  1
 162  0  0  0   1  |  1    1     1      1    |  1    1     1      1    |  0
 170  1  0  0   1  |  1    1     1      1    |  1    1     1      1    |  0
 172  1  0  0   1  |  0    0     0      0    |  1    1     1      1    |  0
 180  0  0  0   1  |  0    0     0      0    |  1    1     1      1    |  0
 182  0  0  0   1  |  0    0     0      0    |  0    0     0      0    |  0
 190  1  0  0   1  |  0    0     0      0    |  0    0     0      0    |  0
 200  0  0  0   1  |  0    0     0      0    |  0    0     0      0    |  0
```

## References


- Assignment problem statements: System Verilog Assignment slides (Siemens, ISI 2026).
- This assignment was made using Claude (an AI assistant by Anthropic). The Verilog code was simulated with Icarus Verilog and the outputs shown are from those runs.